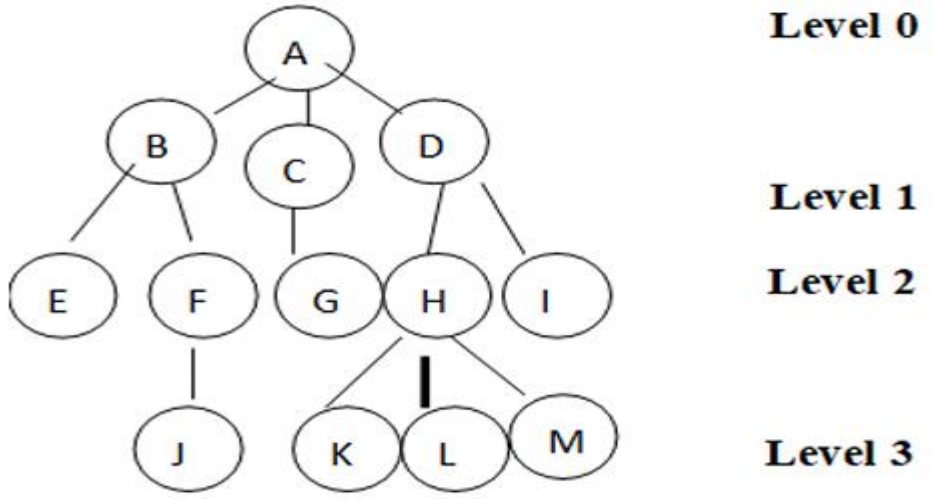
UNIT 4

CHAPTER 1:Trees

DEFINITION:

Tree is non-linear data structure composed of nodes connected by edges,where one node is designated as the root,and every other node is connected to it through parent-child relationships.



Tree Terminologies

1. Root: it is the first node in the hierarchical arrangement of data items. In the above tree, A is the root item.

2. Node: each data item in a tree is called a node. It specifies the data information and links to other data items. There are 13 nodes in the above tree.

3. Degree of a node: it is the number of subtrees of a node in a given tree. In the above tree or the total number of children of a node is called as DEGREE of that Node

a. The degree of node A is 3

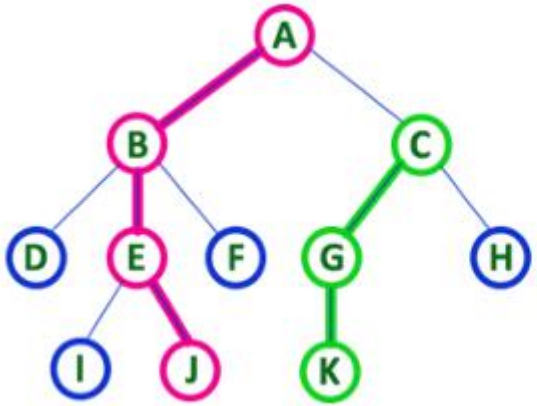
b. Degree of C is 1

c. Degree of D is 2

d. Degree of H is 3

e. Degree of I is 0

- **Terminal node(leaf node):** A node with no children.A node with degree zero is called a terminal node or a leaf. In the above tree, there are 7 terminal nodes. They are E,G,I,J,K, L and M.
- **Non terminal Nodes:** any node except the root node whose degree is non zero is called non terminal node. There are 5 non terminal nodes in the above tree. They are B,C,D,F and H.
- **Siblings:** the children nodes of a given parent node are called siblings. They are also called brothers. In the above tree, E & F are siblings of parent node B. K,L and M are siblings of parent node H.
- **Level:** the entire tree structure is leveled in such a way that the root node is always at level 0, then the intermediate children are at level1 and their intermediate children are at level2 and so on. The above tree, there are 4 levels.
- **Edge:** it is the connecting line of 2 nodes. That is the line drawn from one node to another node is called an edge.
- **Path:** it is the sequence of consecutive edges from the source node to the destination node. In the above tree, the path between A and J is given by the node pairs (A,B), (B, F) and (F, J)

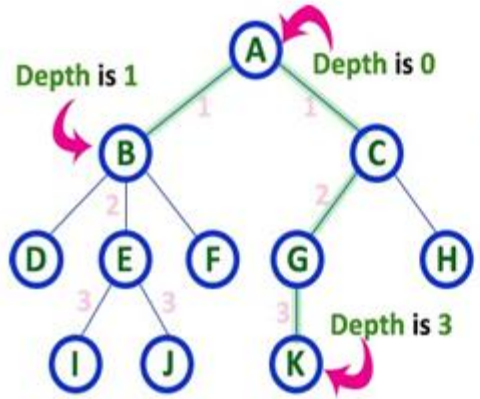


- In any tree, 'Path' is a sequence of nodes and edges between two nodes.

Here, 'Path' between A & J is
A - B - E - J

Here, 'Path' between C & K is
C - G - K

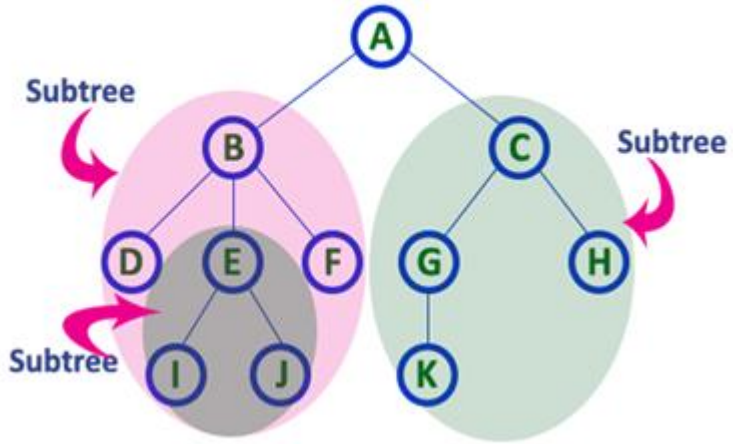
Depth: it is the maximum level of any node in a given tree. In the above tree, the root node A has the maximum level. The term height is also used to denote the depth



Here Depth of tree is 3

- In any tree, 'Depth of Node' is total number of Edges from root to that node.
- In any tree, 'Depth of Tree' is total number of edges from root to leaf in the longest path.

- **Forest:** it is the set of disjoint trees. In a given tree, if you remove its root node then it becomes a forest. In the above tree, there is a forest with 3 trees.
- **Subtree:**In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.

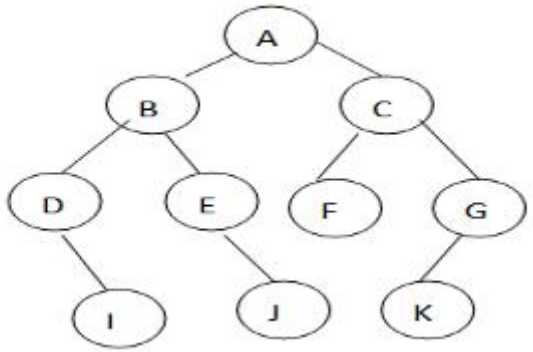


- **Internal node:**In a tree data structure, the node which has atleast one child is called as INTERNAL Node.In a tree data structure, nodes other than leaf nodes are called as Internal Nodes.

- **Ancestors:**All nodes along the path from a node to the root(excluding the node itself)
- **Descendants:**All nodes that are below a specific node in the tree.
- **Degree:**The no.of children a node has.

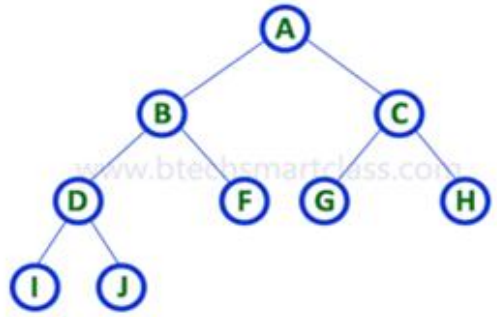
Binary tree

- A **binary tree** is a type of data structure where each **node** has at most **two children**, referred to as the **left child** and the **right child**.
- There may be a zero degree node or one degree node and two degree node.

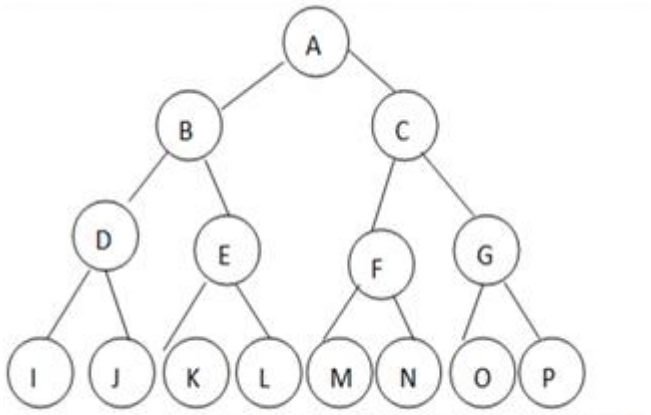


- There are different types of binary trees and they are..

1. **Strictly Binary Tree**: A binary tree in which every node has either two or zero number of children is called Strictly Binary Tree.



2. **Complete Binary Tree**: In a complete binary tree, there is exactly one node at level 0, 2 nodes at level 1, 4 nodes at level 2 and so on.



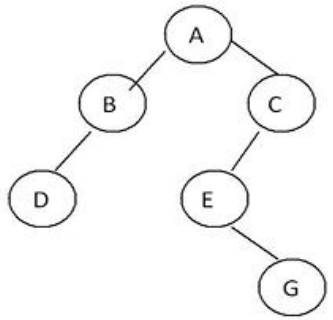
A complete binary tree with depth 4

3.Extended binary tree

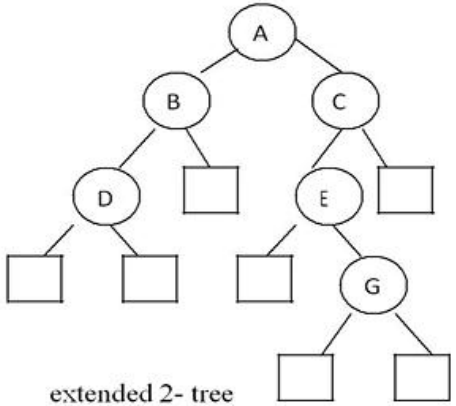
An **extended binary tree**, also called a **2-tree**, is a binary tree in which every node has either **0 or 2 children**.

- **Internal nodes** have **2 children**.
- **External nodes (leaves)** have **0 children**.

Any binary tree can be converted into an extended binary tree by replacing every missing (null) child with an **external node**. This ensures all nodes have either 0 or 2 children.



Binary tree T



extended 2- tree

The nodes are distinguished in diagrams by using circles for internal nodes and squares for external nodes.

Representing Binary Tree in memory

- Let T be a Binary Tree. There are two ways of representing T in the memory as follow
  1. Sequential Representation of Binary Tree.
  2. Link Representation of Binary Tree.

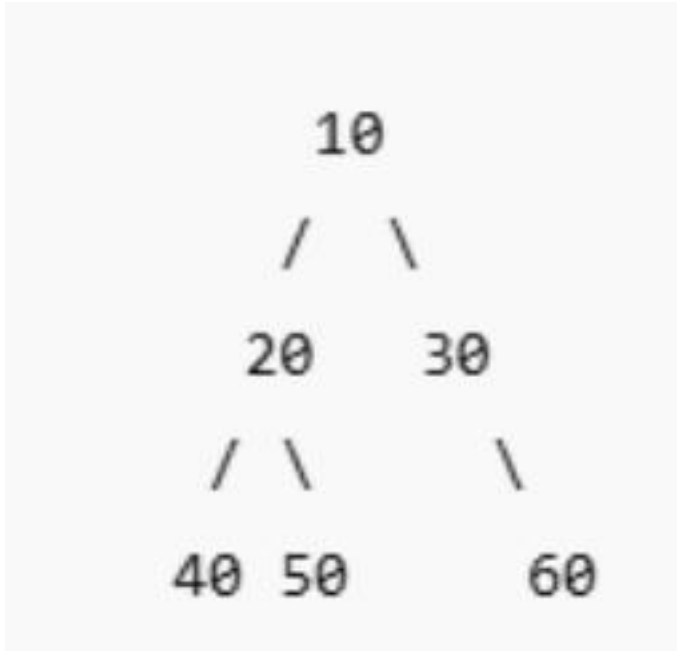
Sequential Representation of Binary Tree.

The tree is stored in an array called TREE[] , where:

The **root node** is stored at index 1 i.e. TREE[1]

For any node at position k :

- Its **left child** is stored at TREE[2 * k]
- Its **right child** is stored at TREE[2 * k + 1]
- Its **parent** (if k ≠ 1) is at TREE[k // 2]

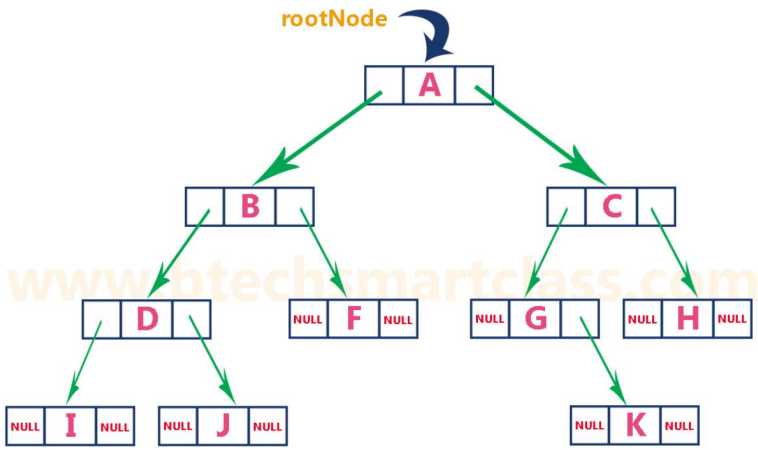


Index: 1 2 3 4 5 6 7
Value: 10 20 30 40 50 None 60
If the root is at index 1:

Left child of node at index k → 2*k
Right child of node at index k → 2*k + 1

Link Representation of Binary Tree

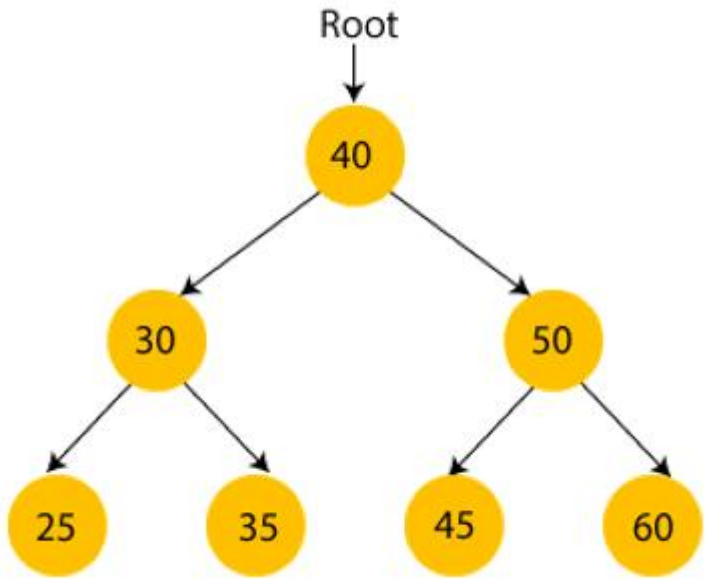
- We use double linked list to represent a binary tree.
- In a double linked list, every node consists of three fields.
- The basic component to be represented in a binary tree is a node. The node consists of 3 fields such as
  1. Data
  2. Left child
  3. Right child.
- The data field holds the value to be given. The left child is a link field which contains the address of its left node and the right child contains the address of its right node



Binary Search Tree

A binary search tree follows some order to arrange the elements.

- In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node.



Traversal of binary tree

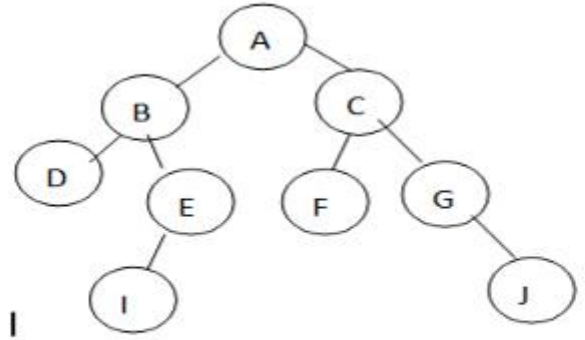
- The traversal is one of the most common operations performed on tree data structure. It is a way in which each node in the tree is visited exactly once in a systematic manner.
- There are 3 popular ways of binary tree traversal. They are
  1. Preorder traversal
  2. Inorder traversal
  3. Postorder traversal

Preorder traversal

- The preorder traversal of a non-empty binary tree is defined as follows.
  1. Visit the root node
  2. Traverse the left subtree in preorder
  3. Traverse the right subtree in preorder
- In a preorder traversal the root node is visited before traversing its left and right subtrees. After visiting the root node, the left subtree is taken up and it is traversed recursively, then the right subtree is traversed recursively.
- **Algorithm for preorder order traversal**

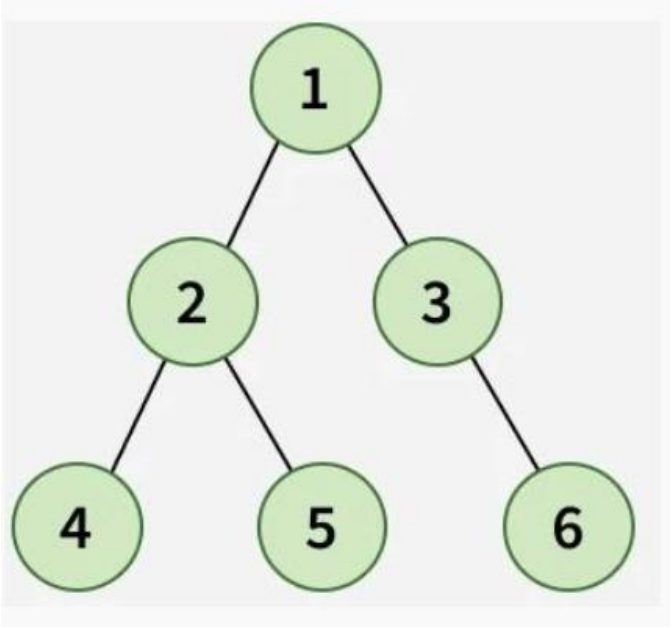
Procedure Preorder(root)

1. If (root!=NULL) Begin
2. Print root->info
3. Preorder(root->left)
4. Preorder(root->right) End
5. finished



The preorder traversal of the above binary tree is ABDEICFGJ

Qn:Write the preorder for the given tree.

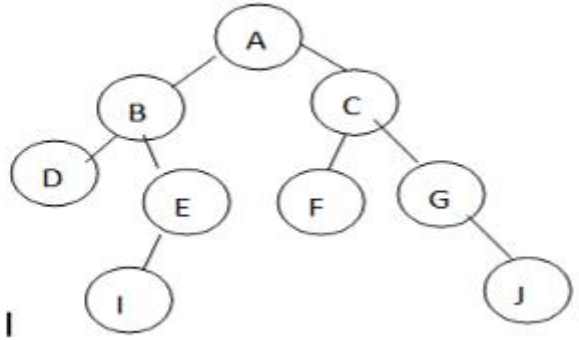


Inorder Traversal

- The inorder traversal of a non-empty binary tree is defined as follows.
  1) Traverse the left subtree in Inorder
  2) Traverse the root node
  3) Traverse the right subtree in Inorder
- In an Inorder traversal, the left subtree is traversed recursively before visiting the root node. After visiting the root node, the right subtree is taken up and it is traversed recursively.

Procedure Inorder(root)

1. If (root !=NULL) Begin
2. Inorder(root->left)
3. Print root->info
4. Inorder(root->right) End
5. finished



The Inorder traversal of the above binary tree is DBIEAFCGJ

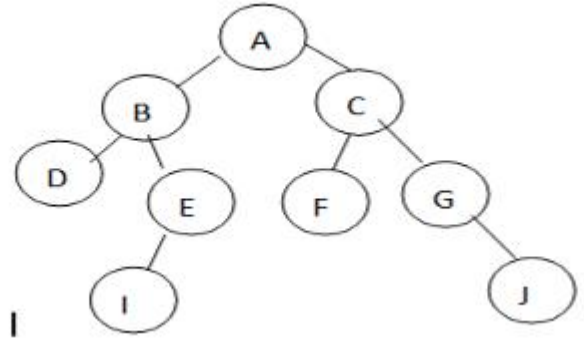
Postorder traversal

- The Postorder traversal of a non-empty binary tree is defined as follows.
  1) Traverse the left subtree in Postorder
  2) Traverse the right subtree in Postorder
  3) Traverse the root node

In a postorder traversal the left and right subtree is traversed recursively before visiting the root node. The left subtree is taken up and it is traversed recursively, then the right subtree is traversed recursively. finally root node is visited.

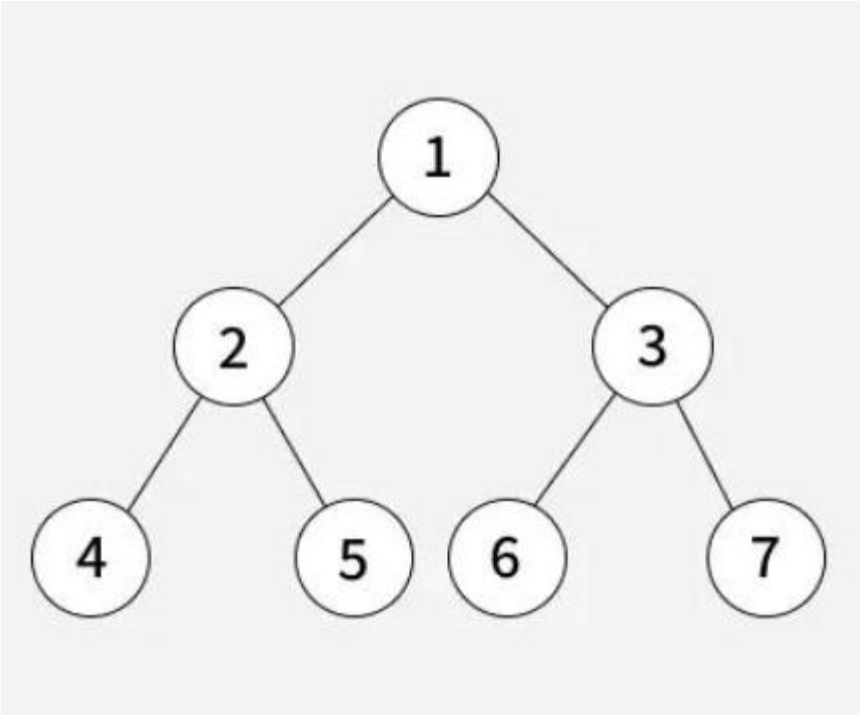
Algorithm for postorder order traversal

Procedure postorder(root)
1. If (root !=NULL) Begin
2. postorder(root->left)
3. postorder(root->right)
4.Print root->info
5.End
5.finished



The Postorder traversal of the above binary tree is DIEBFJGCA

Find the preorder, inorder, and postorder traversals of the given tree.



Inorder Traversal

4	2	5	1	6	3	7

Preorder Traversal

1	2	4	5	3	6	7

Postorder Traversal

4	5	2	6	7	3	1

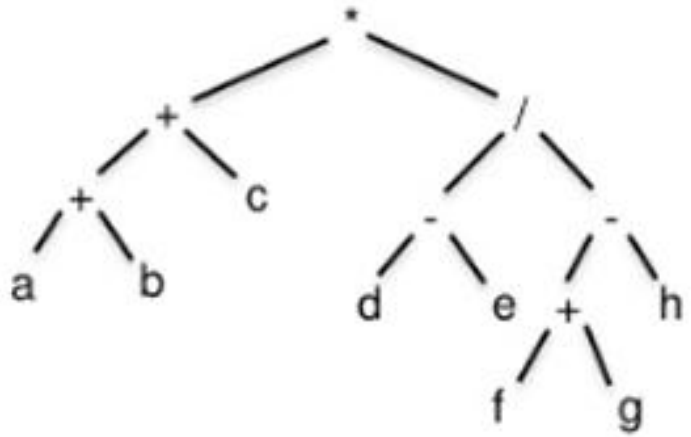
Draw the binary tree for the following

INORDER: E, A, C, K, F, H, D, B, G

PREORDER: F, A, E, K, C, D, H, G, B

Expression Tree:

- It is a binary tree, which stores an arithmetic expression. The leaf nodes of the expression tree are always operands and all other internal nodes are the operators, including the root. Divide and conquer technique used to convert an expression into a binary tree.
- E.g. (a + b + c) * ((d – e) / (f + g – h))



- Precedence Table:

Brackets innermost: Highest level

^ (exponential) : Higher level

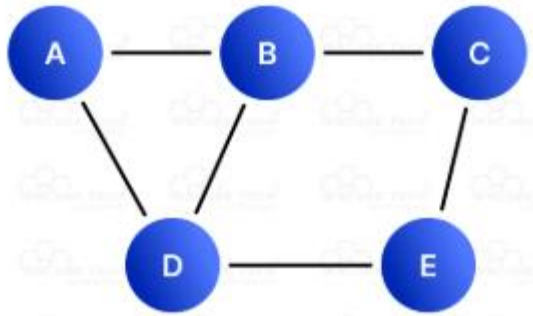
* and / : Middle level

+ and - : Lowest level

Graphs

A **graph** is a **non-linear data structure** that consists of:

- A set of **vertices** (also called **nodes**), and
- A set of **edges** that connect pairs of vertices.



Graph Terminologies

- **Vertex :**

A vertex (plural: vertices) is a fundamental unit in a graph that represents an entity or a point. The set of all vertices in a graph is denoted by V.

- **Edges:**

An edge is a connection between two vertices. It represents the relationship or link between them. The set of all edges is denoted by E.

  - Directed Edge: An edge with a direction, represented as an ordered pair (u,v), where u is the starting vertex, and v is the ending vertex.
  - Undirected Edge: An edge with no direction, represented as an unordered pair {u,v}.

Degree of a Vertex :

The degree of a vertex is the number of edges connected to it.

- In-degree: The number of edges directed toward a vertex (only for directed graphs).
- Out-degree; The number of edges directed outward from a vertex (only for directed graphs).
- Total Degree: Sum of the in-degree and out-degree of a vertex in a directed graph.

- **Adjacent node**

If two nodes share an edge, they are **adjacent** to each other.

Types of Graphs:

- **Directed Graph (Digraph)**: Edges have direction.
- **Undirected Graph**: Edges have no direction.
- **Weighted Graph**: Edges have weights (costs, distances, etc.).
- **Unweighted Graph**: Edges are all equal in value.

Representations of Graph

- Here are the two most common ways to represent a graph:

1. Adjacency Matrix
2. Adjacency List

Adjacency Matrix

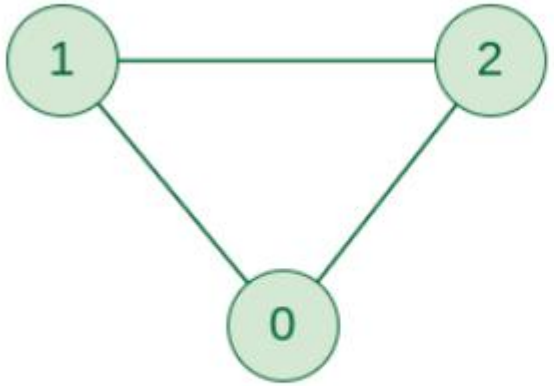
An adjacency matrix is a way of representing a graph as a matrix of boolean (0's and 1's)

Let's assume there are **n** vertices in the graph So, create a 2D matrix **adjMat[n][n]** having dimension n x n.

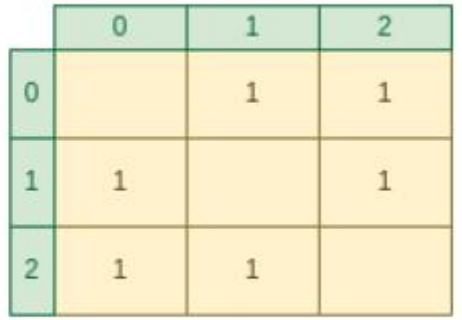
- *If there is an edge from vertex **i** to **j**, mark **adjMat[i][j]** as **1**.*
- *If there is no edge from vertex **i** to **j**, mark **adjMat[i][j]** as **0**.*

Representation of Undirected Graph as Adjacency Matrix:

The entire Matrix is initialized to 0. For **undirected graphs**, the matrix is **symmetric**, so if i is connected to j , then both adjMat[i][j] = 1 and adjMat[j][i] = 1



Undirected Graph

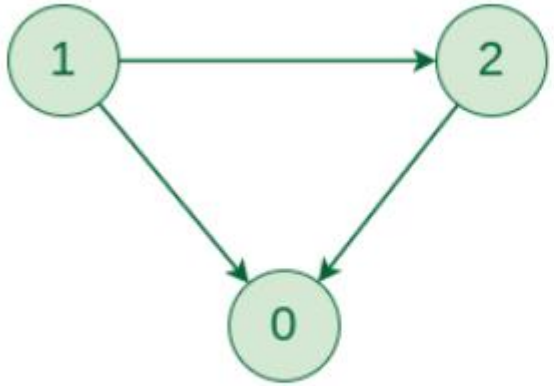


Adjacency Matrix

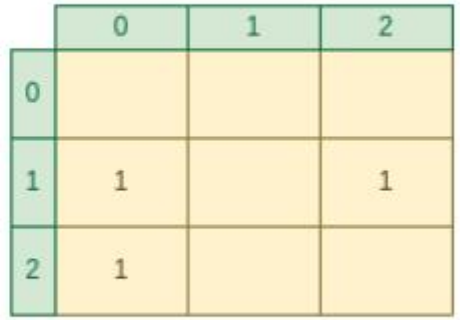
Representation of Directed Graph as Adjacency Matrix:

Initially, the entire matrix is initialized to **0**. If there is an edge from source to destination,

we insert **1** for that particular **adjmat[destination]**.



Directed Graph



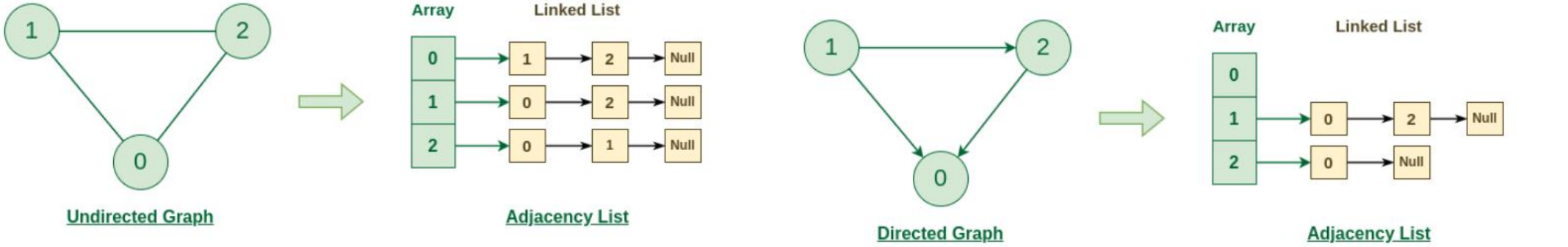
Adjacency Matrix

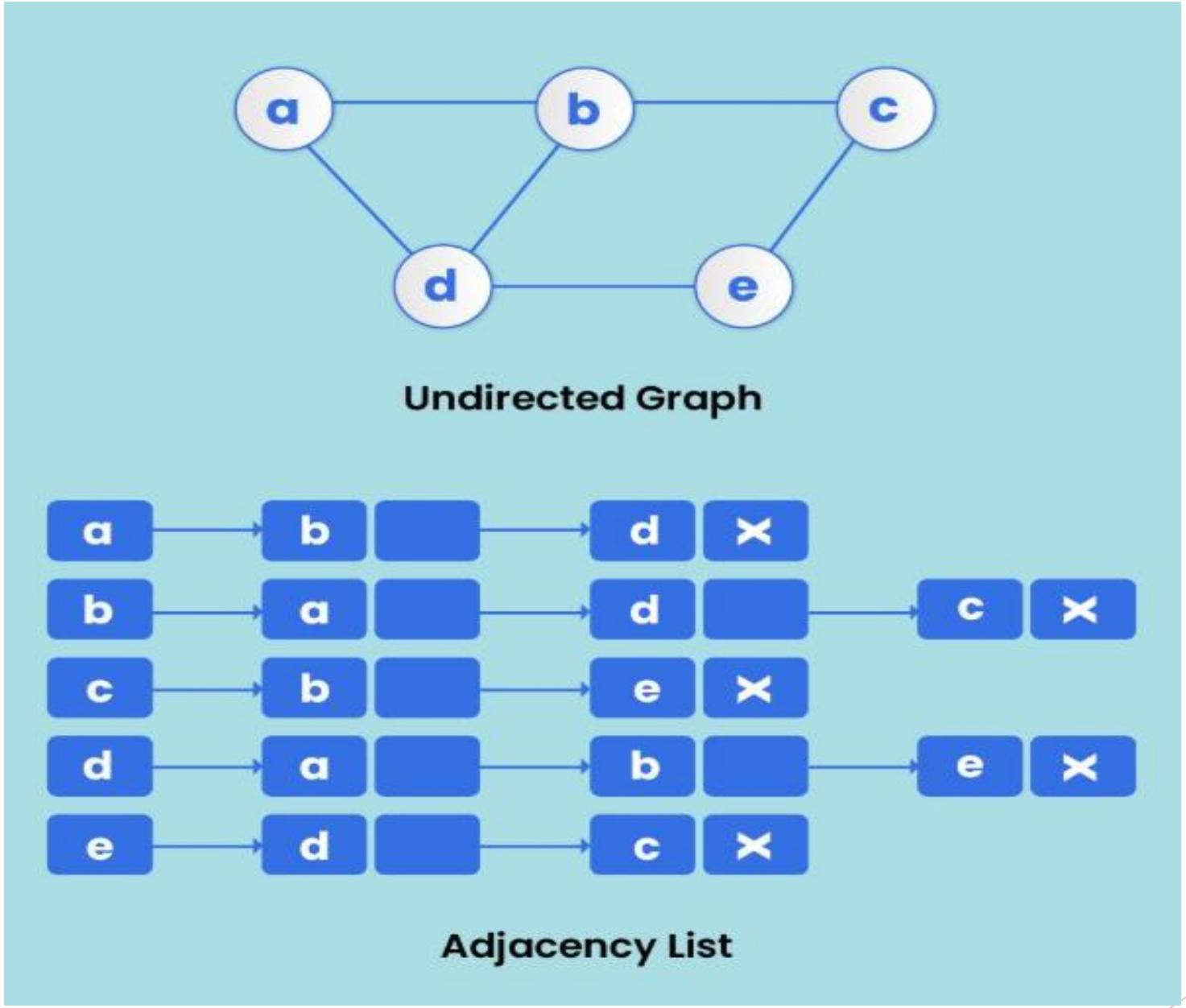
linked representation of a graph

- **Linked representation of a graph**, also known as an **adjacency list.**
- **Each vertex** has a list of its **neighbors** (the vertices it is directly connected to).

These lists are stored in something like an **array or a list**, where:

- Each position in the array represents a vertex.
- At each position, there's a **linked list** (or just a list) of other vertices that are connected to it.



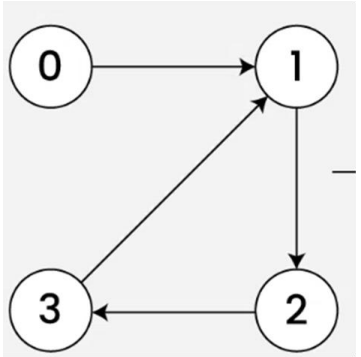

a
b
c
d
e
Undirected Graph
a
b
d
b
a
d
c
c
b
e
d
a
b
e
e
d
c
Adjacency List

Path Matrix

- The **path matrix**, also called the **reachability matrix** .
- **Definition:** For a graph with n vertices, the path matrix P is an n×n boolean matrix where:

$$P[i][j] = \begin{cases} 1 & \text{if there is a path (direct or indirect) from vertex } i \text{ to vertex } j \\ 0 & \text{otherwise} \end{cases}$$

- **For Directed Graphs**: The direction of edges is respected. So $P[i][j] = 1$ only if there's a sequence of directed edges leading from i to j.
- **For Undirected Graphs**: The matrix is symmetric since paths can be traversed in both directions.



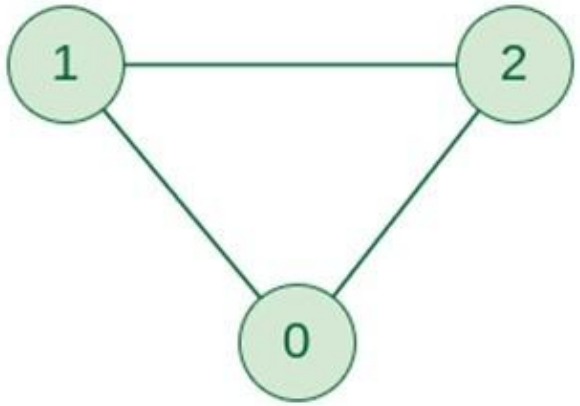
Path Matrix

	0	1	2	3
0	0	1	1	1
1	0	1	1	1
2	0	1	1	1
3	0	1	1	1

Adjacency Matrix

- An adjacency matrix is a way of representing a graph as a matrix of Boolean (0's and 1's)
- Let's assume there are **n** vertices in the graph So, create a 2D matrix **adjMat[n][n]** having dimension n x n.
- If there is an edge from vertex **i** to **j**, mark **adjMat[i][j]** as **1**.
- If there is no edge from vertex **i** to **j**, mark **adjMat[i][j]** as **0**.

Representation of Undirected Graph as Adjacency Matrix:



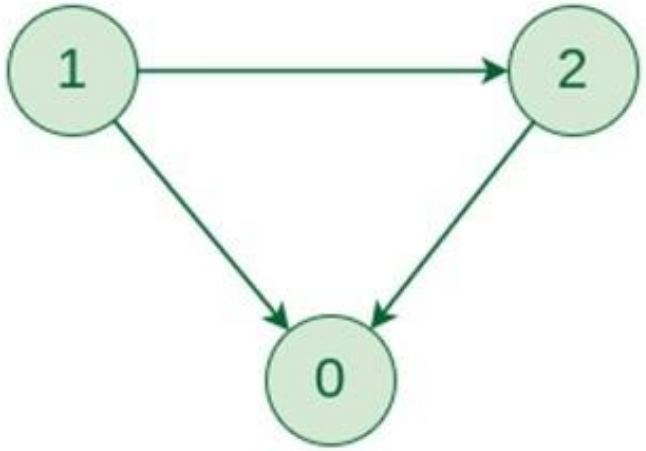
Undirected Graph

	0	1	2
0		1	1
1	1		1
2	1	1	

Adjacency Matrix

Graph Representation of Undirected graph to Adjacency Matrix

Representation of Directed Graph as Adjacency Matrix:



Directed Graph

	0	1	2
0			
1	1		1
2	1		

Adjacency Matrix

Graph Representation of Directed graph to Adjacency Matrix

Breadth-First Search (BFS)

Explore the graph level by level, starting from the source node. It visits all the neighbors of a node before moving on to the next level of neighbors.

Data structure used: Queue (FIFO)

Steps:

1. Start from a source node.
2. Visit it and mark it as visited.
3. Enqueue all its unvisited neighbors.
4. Dequeue a node and repeat the process.

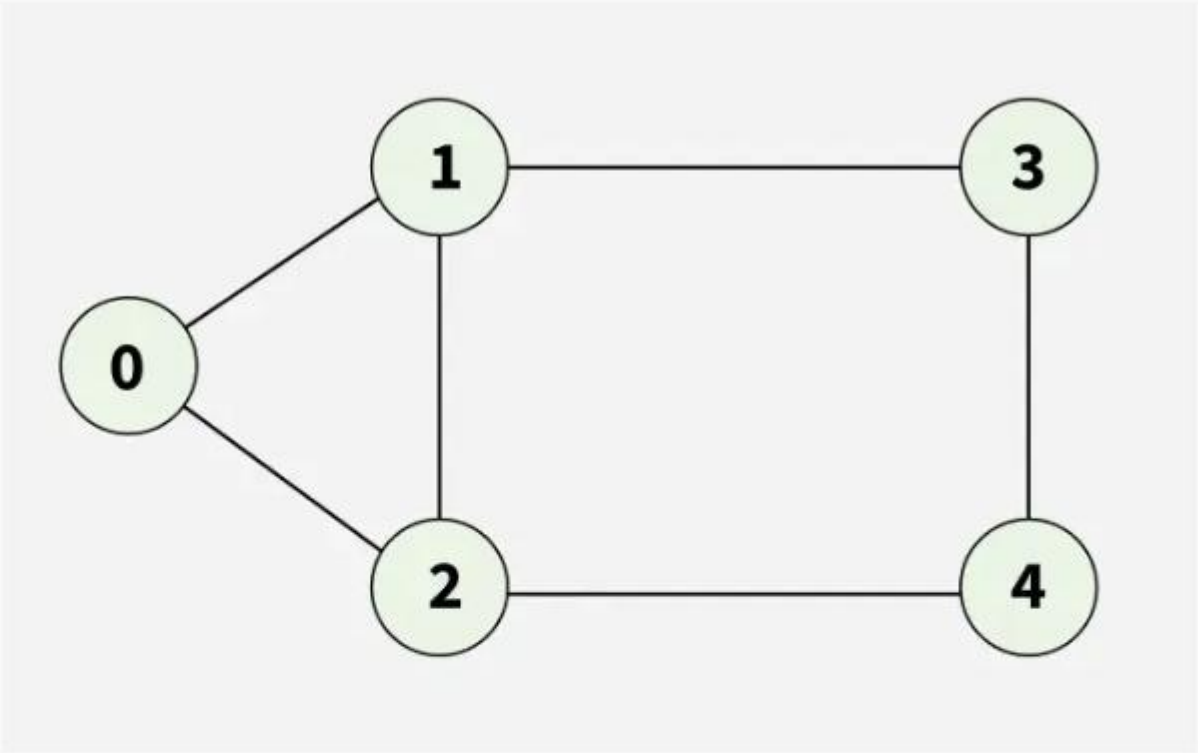
BFS Algorithm:

This algorithm executes a BFS search on a graph G beginning at a starting node A.

1. Initialize all nodes to the ready state i.e STATUS=1
2. Put the starting node A in QUEUEU and change its status to the waiting state. (status =2)
3. Repeat steps 4 and 5 until QUEUE is empty.
4. Remove the front node N of QUEUE. Process n and change the status of n to the processed state(STATUS=3)
5. Add to the rear of QUEUE all the neighbors of N that are in the ready state and change their status to the waiting state

   End of loop

6.Exit



Output: *[0, 1, 2, 3, 4]*
Explanation: *Starting from 0, the BFS traversal will follow these steps:*
Visit 0 → Output: [0]
Visit 1 (first neighbor of 0) → Output: [0, 1]
Visit 2 (next neighbor of 0) → Output: [0, 1, 2]
Visit 3 (next neighbor of 1) → Output: [0, 1, 2, 3]
Visit 4 (neighbor of 2) → Final Output: [0, 1, 2, 3, 4]

Depth-First Search (DFS)

Explore as far as possible along one branch before backtracking.

Data structure used: Stack (LIFO) or recursion (implicit stack)

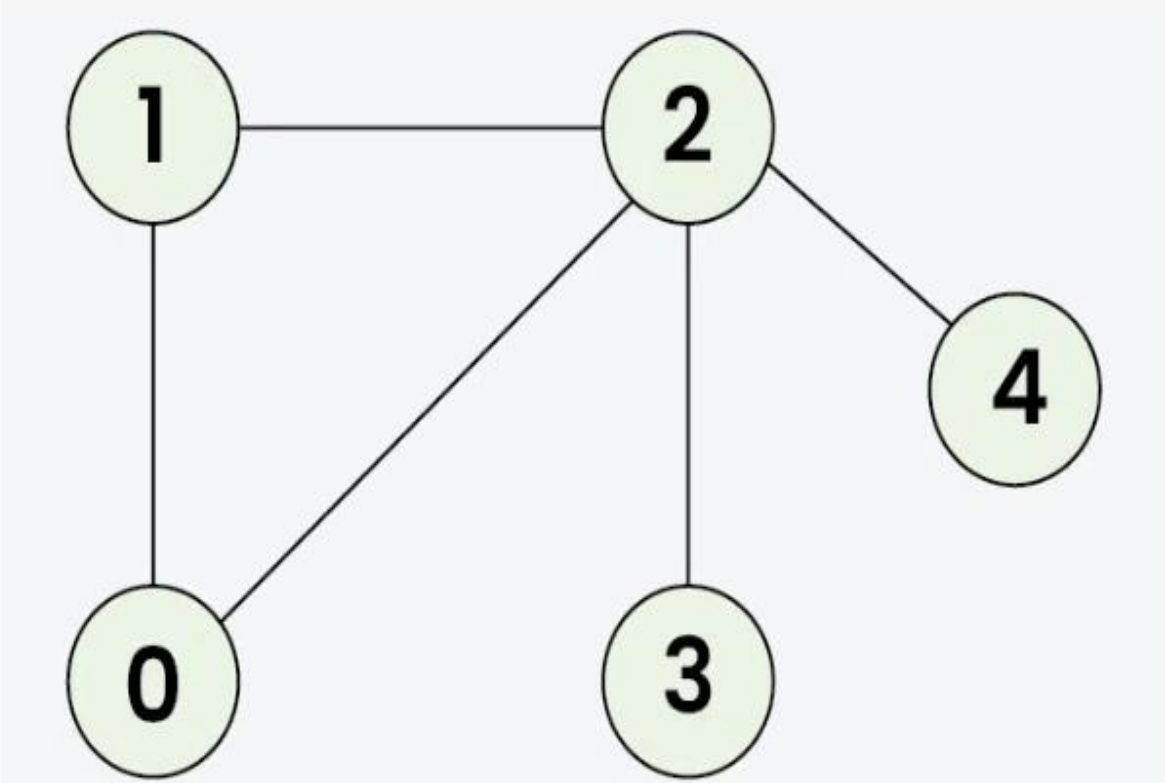
Steps:

1. Start from the source node.
2. Visit it and mark it as visited.
3. For each unvisited neighbor, recursively apply DFS.

DFS Algorithm

This algorithm executes a depth first search on a graph G beginning at a starting node A.

1. Initialize all the nodes to the ready state (status=1)
2. Push the starting node onto stack and change its status to the waiting state (status=2)
2. Repeat steps 4 and 5 until stack is empty.
3. Pop the top node N of stack. Process N and change its status to the processed state(status=3)
4. Push onto stack all the neighbors of N that are still in the ready state(status=1), and change their status to the waiting state (status=2)
   End of step 3 loop
5. Exit



Output: *[0 1 2 3 4]*
Explanation: *The source vertex s is 0. We visit it first, then we visit an adjacent.*
Start at 0: Mark as visited. Output: 0
Move to 1: Mark as visited. Output: 1
Move to 2: Mark as visited. Output: 2
Move to 3: Mark as visited. Output: 3 (backtrack to 2)
Move to 4: Mark as visited. Output: 4 (backtrack to 2, then backtrack to 1, then to 0)